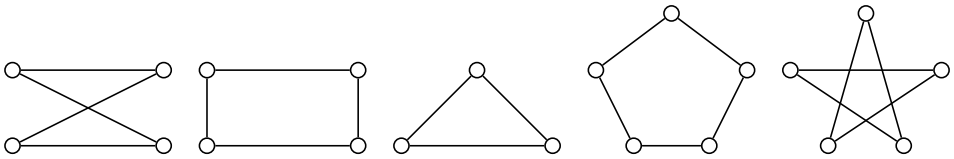
TEN USES OF FLAGCALC FOR THE CLASSROOM

PETER GLENN

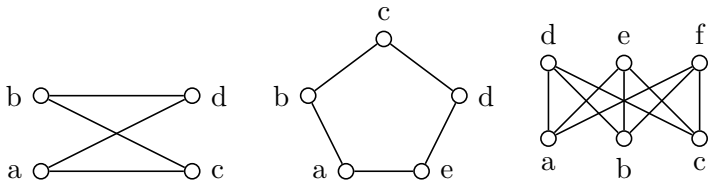
1. Introduction

Welcome to a world of discovery: the advent of computer-assisted proof and mathematical exploration has produced many tools, and the present paper focuses on just one tool, open-source graph-theoretic command-line tool *Flagcalc*, from ten perspectives. Many of these perspectives also mesh nicely with Flagcalc's GUI (Graphical User Interface) front-end called *Explore*. Indeed, the versatility of these complementary tools makes any exposition a bit tricky to design: all manner of inventiveness is possible.

Prior knowledge of *simple graphs* is briefly reviewed here: a simple graph is a set V of vertices (finite) and a set E of edges between any two distinct vertices. Here are five example graphs.



The small circles are vertices; the straight lines are edges. Often we will label the vertices with lower-case letters, as in the three graphs below:



The tool happily generates random graphs or procures them from included and user-created graph databases. Then, one can look at the graphs from several perspectives, most notably through the built-in "first-order" querying language *Flagcalc Querying Language*, or built-in features around sorting graphs and simpler queries that just assess one *measure* or another, including those user-defined via *stored procedures* or Python extensions.

Always, as a command-line tool, the invocation of help, via the argument `./flagcalc -h` produces a long list of options for the user to peruse.

Date: August 22, 2026.

2. One: Procuring Graphs to Work On

Two necessary skills are Flagcalc's *verbosity* settings, and its *workspace*. Graphs pile up on the workspace, along with associated computations nicely stored and categorized. Then, the verbosity requests explain to Flagcalc what to tell us about the workspace, i.e. "Please list all the graphs on the workspace", or "Summarize the percentage of graphs on the workspace that passed a given test". Clearly verbosity is very important: you might have tested a thousand graphs for a property, and for the tool to list all one thousand graphs would be overwhelming to the user, but to get the summary percentages or average values, max and min, etc., is most helpful.

2.1. **Populate the Workspace with Graphs.**

2.1.1. *Using Flagcalc's Graph Format.* Start Flagcalc with the option

```
./flagcalc -d std::cin
```

and you will be able then to type in your favorite graph. To tell the parser you are done typing it, you type out `END END` or `### ###`.

If your vertices are labeled with lower-case letters of the alphabet then typing them adjacent to each other serves to create an edge between them. Typing just a vertex alone introduces the vertex, as does using it in an edge; you see you might have an isolated vertex not connected to the rest of the graph, so sometimes you just have say an `a` alone. So, the K_3 graph pictured earlier (the triangle) could be written as

```
ab bc ca
```

I.e. via its three edges. Or if you group several vertices together without spaces, it creates that clique; K_3 a second time is written as

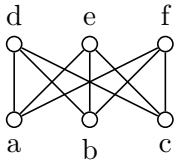
```
abc
```

Finally, a *path* can be written preceded by a dash, so K_3 again is

```
-abca
```

which actually is a cycle, a special kind of path that begins and ends at the same vertex. Indeed the dash formalism extends beyond paths to include *walks*, that is, it may revisit its own interior vertices along the way (which a path technically cannot do).

Moving along, we also have what are called bipartite graphs, the example being the $K_{3,3}$ graph featured here:



Again, there are several ways to express this graph in Flagcalc: first,

```
abc=def
```

The equal sign simply means make every possible connection between the vertices on the left of the equal sign to those on the right. Or,

```
ad ae af bd be bf cd ce cf
```

works just fine, as does a fancy way:

```
abcdef !abc !def
```

The six letters on the left form a clique of size six; then the exclamation point removes the two K_3's, i.e. removes edges `ab ac bc` and edges `de df ef`.

2.1.2. *Using a Graph Database.* Probably you'd like to use a text editor and create a whole set of graphs to work on. Just separate each graph with the same `END END` or `### ###`, and your homemade database can number from the dozens to the millions. To load the graphs into Flagcalc's workspace, invoke

```
./flagcalc -d <filename.fcg>
```

2.1.3. *Using Random Graphs.* More likely, you just want some material to work with, and perhaps to search high and low for graphs that meet your user-specified *criteria* or *measures* as explained at length below. So the populating of the workspace, is just from randomly chosen graphs.

```
./flagcalc -r 10 p=0.5 100
```

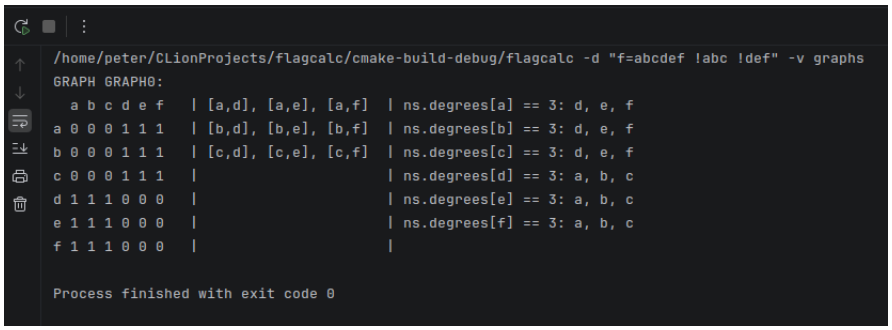
This produces one hundred random graphs on ten vertices with a likelihood statistically of 1/2 of an edge between any two distinct vertices.

2.2. **Summarizing the Graphs with Verbosity.** Flagcalc parses the command line arguments in the order they are given. Usually therefore verbosity is the last one. It tells it what and how to print out the results. For small workspaces often you want the graphs themselves:

```
./flagcalc ... -v graphs
```

Or, you may want a summary about the *queries* (criteria and measures) you have applied; in that case, use the built-in file below to list out all the verbosities requested (you can inspect this file in a text editor if you want to get under the hood):

```
./flagcalc ... -v i=minimal3.cfg
```



Along the way you see yet another way to input a graph: instead of `-d std::cin` you can use

```
-d f="<graph spec>"
```

or several such `f=`'s. Note in the screenshot above, on the left is what's called an *adjacency matrix*; in the middle is the list of edges; and on the right is the degree and neighbors of each vertex.

3. Two: Some Graph Characteristics

Does a graph contain every possible edge? Then it is *complete*. Does a graph have a path between any two vertices? Then it is *connected*. Does a graph have the same *degree* for every vertex (where *degree* is the number of edges attached to the given vertex)? Then it is *regular*.

3.1. **Complete Graphs.**

The complete graphs on n vertices are so special they have a name: K_n. In the eight graphs that opened this paper, only K_3 was a complete graph. If a graph has a subgraph that is complete, the subgraph is called a *clique*. (A *subgraph* is a subset U of the vertex set V, along with some subset of the edges whose ends are both in U; a subgraph is *induced* if it contains all such edges).

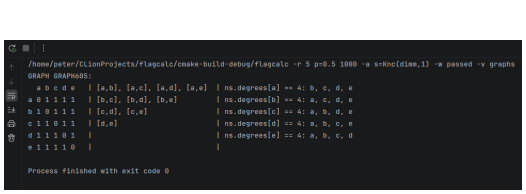
To test if a graph is complete, use

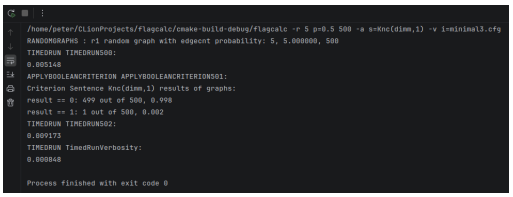
```
./flagcalc ... -a s="cliquem == dimm" ...
```

This checks whether the graph's largest clique (complete subgraph) has as many vertices as the graph's *dimension*. Alternately,

```
./flagcalc ... -a s="Knc(dimm,1)" ...
```

This checks that the graph has at least one clique of size equal to its dimension. Here are two similar invocations, with differing verbosities





Note a complete graph on five vertices has $\binom{5}{2} = 10$ possible edges; $(\frac{1}{2})^{10} = \frac{1}{1024}$, so we expect on average about one out of a thousand such random graphs to be complete, i.e. K_5.

Another way to test for a graph being complete is to check if it *embeds* the complete graph K_n:

```
./flagcalc -r 5 p=0.5 1024 -a s=embedsc(\"abcde\") ...
```

The backslashes preceding the internal quotation marks are necessary.

3.2. **Hamiltonian Graphs.**

A graph is *Hamiltonian* if it has a cycle encompassing every vertex[1]. This is most intuitively coded below on the second line of Flagcalc invocation,

[1]Please see the shell script `scripts/testsuitesimple.sh` lines one through thirty-two for an intro to Hamiltonian graphs in Flagcalc, or the pdf located at `https://github.com/corralledcode/flagcalc/blob/main/documentation/flagcalcintro.pdf`.

where we use `st(c) == dimm`, i.e. the "size tally" of the cycle c is equal to the graph's dimension measure (`dimm`).

```
PTH=${PTH:-'../bin'}

# Hamiltonian cycles, in four distinct ways
$PTH/flagcalc -r 8 p=0.5 100 -a s="EXISTS (p IN Perms(V), FORALL (a IN dimm, ac(p[a],p[(a+1) % dimm])))" all -v i=minimal3.cfg
$PTH/flagcalc -r 8 p=0.5 100 -a s="EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
$PTH/flagcalc -r 8 p=0.5 100 -a s="EXISTS (p IN Perms(V), FORALL (a IN dimm, ac(p[a],p[(a+1) % dimm])))" s="EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
$PTH/flagcalc -r 8 p=0.5 100 -a s="EXISTS (p IN Perms(V), FORALL (a IN dimm, ac(p[a],p[(a+1) % dimm]))) IFF EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
$PTH/flagcalc -r 8 p=0.5 100 -a s="embedsgenerousc(\"-abcdefgha\")" all -v i=minimal3.cfg
$PTH/flagcalc -r 8 p=0.5 100 -a s="embedsgenerousc(\"-abcdefgha\") IFF EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
$PTH/flagcalc -r 8 p=0.5 100 -a s="circm == dimm" all -v i=minimal3.cfg
$PTH/flagcalc -r 8 p=0.5 100 -a s="circm == dimm IFF EXISTS (c IN Cycless(0), st(c) == dimm)" all -v i=minimal3.cfg
```

The last equivalent means of expressing Hamiltonicity is the most succinct: it uses the measure `circm` which is the graph's *circumference*, defined to be the length of its largest cycle.

As an exercise, simply try and produce a Hamiltonian graph on a dimension of your choice (e.g. try the graph `abcde -efghd`. Then use Flagcalc to see if it is Hamiltonian:

```
./flagcalc -d f="abcde -efghd" -a s="circm == dimm" -v i=minimal3.cfg
```

Alternately, do the reverse: ask Flagcalc for a few graphs that are Hamiltonian (as found in a large-enough subset of random graphs), and verify them by hand:

```
./flagcalc -r 10 p=0.5 100 -a s="circm == dimm" -v crit min any
```

3.3. **Graphs Admitting an Euler Tour.** An Euler tour is a graph which has a *walk* (that is, a sequence of steps between adjacent vertices, including steps that self-intersect the walk) that crosses every edge. In the section on stored procedures below, it is shown how to make a stored procedure called "Eulertourc". We will postpone that detailed intro, and for now just use the feature "load all stored procedures" followed by a simple query:

```
./flagcalc -r 10 p=0.5 100 -a isp=storedprocedures.dat s=Eulertourc -v
                                i=minimal3.cfg
```

As with the Hamiltonian graphs above, one can first try sketching a graph with an Euler tour, then asking Flagcalc to verify; then, alternately, ask Flagcalc to procure some Eulerian graphs, and verify by hand.

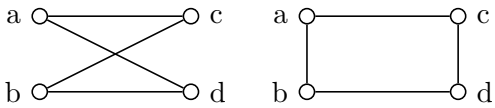
```
/home/peter/CLionProjects/flagcalc/cmake-build-debug/flagcalc -d "f=-abcdaefga -ahijc -ahikf" -a isp=../scripts/storedprocedures.dat s=Eulertourc -v i=minimal3.cfg allsets
Opening file ../scripts/storedprocedures.dat
Opening file ../scripts/recursion.dat
Opening file ../scripts/planarity.dat
Opening file ../scripts/regular.dat
TIMEDRUN TIMEDRUN1:
0.000312
APPLYBOOLEANCRITERION APPLYBOOLEANCRITERION2:
Criterion Sentence Eulertourc results of graphs:
result == 0: 1 out of 1, 1
TIMEDRUN TIMEDRUN3:
57.7248
TIMEDRUN TimedRunVerbosity:
7.3e-05

Process finished with exit code 0
```

This is the graph of the bridges of Konigsberg that inspired Euler to state his question.

3.4. **Extremal Graphs.**

4. Two: Equivalence Up To Isomorphism

Almost any *criteria* or *measure* of a graph should be the same regardless of what order the vertices are specified in. Therefore the two graphs below are said to be *isomorphic*:



Two features, then: an algorithm called "Flagcalc fingerprinting" quickly decides if two graphs are isomorphic, and also sorts a list or workspace of graphs according to the internal logic of fingerprinting, in a linear fashion. This is the `-f` feature of Flagcalc. Next, the `-i` feature counts how many *automorphisms* a given graph has, i.e. how many ways to make it isomorphic to itself[2].

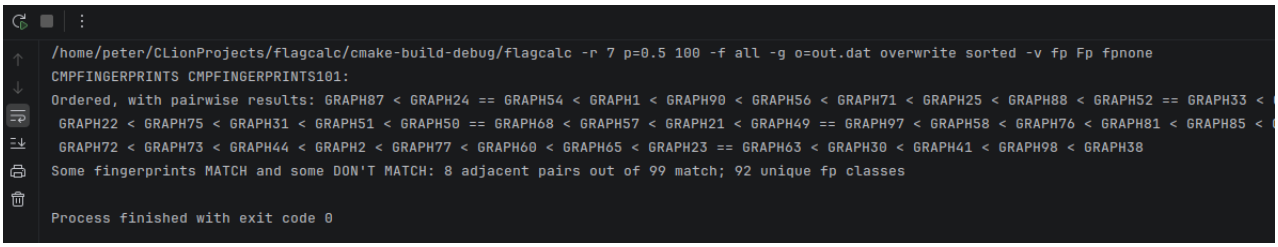
4.1. **Sorting by "Fingerprint".** For example,

```
./flagcalc -r 7 p=0.5 100 -f all -g o=out.dat overwrite sorted -v fp Fp
                                                                fpnone
```

It seems rather extravagant, but first these are the fp verbosities needed, and along the way we see a new feature, `-g` which outputs one graph per equivalence class to the stated database "out.dat", i.e. so no two graphs in out.dat are isomorphic. To check that this is true of out.dat, run

```
./flagcalc -d out.dat -f all -v fp Fp fpnone
```

second, which should show "NO fingerprints match", Flagcalc's way of saying "no two graphs are isomorphic".



4.2. **Counting Automorphisms.**

[2]A more sophisticated tool would go a step further and discuss *automorphism groups*, but for now this isn't implemented in Flagcalc.

The K_5 graph on the left has $5! = 120$ automorphisms, since any permutation of its vertices still matches the same set of edges. The $K_{3,3}$ graph on th eright has $3! * 3! * 2 = 72$ automorphisms. Please experiment with graphs of your own design! For example: true or false: for any natural number n, there is a graph with n automorphisms.

4.3. **Paring Down a Dataset of Graphs: One Representative Per Class.** You can also pare down the output *after* running some queries or such, if your queries are reasonably quick to run, you can use `-f passed` in place of `-f all` to instruct Flagcalc only to sort by fingerprint those graphs that passed the preceding criteria (the `-a` portion we are slowly getting familiar with).

5. Three: Iterated Criteria

Most textbook theorems start with a few conditions: "Provided that the graph is connected, then being a tree is the same thing as being a forest" (that sounds obvious, but think through how to express the conditions of being a tree or being a forest as a logical statement). "Provided that the graph is planar, then four colors suffice to color it so no two adjacent vertices have the same color." "Provided that a graph is triangle-free, then its maximum number of edges is $\lfloor \frac{n^2}{4} \rfloor$."

One can also iterate more than once, and have one sample winnowed down, then another winnowing, and finally the punchline (which in Flagcalc need not itself be boolean like the earlier iterations: it can be a measure (double type), a tally (integer type), or a set type, or a tuple type, or even a graph type).

These are simply done with

```
./flagcalc -r 10 18 2000 -a c=cr1 s2="edgecm > dimm^2/4" all -v
                          i=minimal3.cfg
```

which should return zero out of two thousand (we have just polled two thousand random graphs looking for a counterexample to the theorem—Mantel's Theorem—in the preceding paragraph). Here, `cr1` is the condition of being triangle-free (from a stage of Flagcalc's development when only a handful of criteria were envisioned). Note `c` and `s` are interchangeable, and the important thing is the numeral `2` next to `s`[3].

For the forest and tree relationship, note that a tree is a graph in which between any two vertices there is exactly one path. Alternately, it is a connected graph in which there

[3]Also good to note that triangle-free is equivalent to any number of alternative expressions:

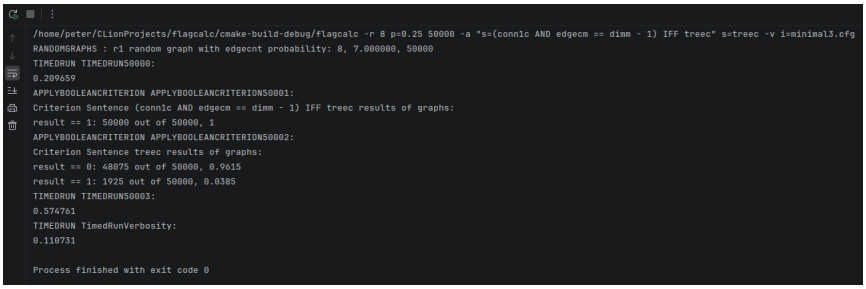
- `NOT embeds("abc")`
- `NOT Knc(3,1)`
- `cliquem < 3`
- `FORALL(a IN V, b IN V, c IN V, NOT (ac(a,b) AND ac(b,c) AND ac(a,c)))`
- `FORALL(C IN Cycless, st(C) > 3)`
- `NAMING (W AS nwalksbetweenp(3), FORALL (v IN V, (W[v])[v] == 0))`
- `NAMING (W AS CUDAnwalksbetweenp(3), FORALL (v IN V, (W[v])[v] == 0))`
- etc.

are no cycles. Thirdly, we might conjecture, a connected graph on n vertices is a tree if it has $n-1$ edges. How would Flagcalc help test this conjecture?

There are several approaches. Best might be

```
./flagcalc -r 8 p=0.25 50000 -a "s=(conn1c AND edgecm == dimm - 1) IFF
                    treec" s=treec -v i=minimal3.cfg
```

Read it together: it takes 50,000 random graphs on 8 vertices and one-quarter edge likelihood, then uses the logical "IFF" ("if and only if") to check that being both connected and having edge count one less than vertex count, is logically equivalent to being a tree. The second invocation of "treec" is just a sanity check: if none of the graphs were trees then it would be a rather lopsided or blind assertion; we indeed see nearly four percent are trees, so that is not insubstantial. The numbers 50,000 and 0.25 are from trial-and-error, starting with, say, 100 graphs, and checking how many graphs the system can process in a reasonable amount of time.



Please note once and for all, that Flagcalc was coded by coders, and "1" is the same thing as "True", and "0" the same as "False". Hence the value "1" for all fifty-thousand random graphs: the hypothesis holds.

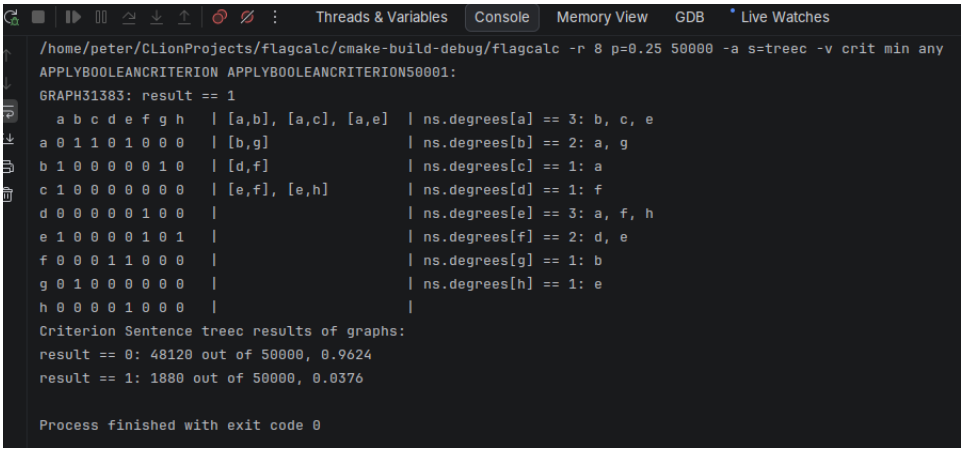
6. Four: Percentages and Likelihoods

This is a great segue to percentages and likelihoods.

7. Five: Small Sample Space

In a small sample space one can really explore notions like planarity, like graph colorings, and so much more via pen-and-paper alongside Flagcalc's curated responses to queries and its presentation of data found in databases of graphs.

E.g. use the verbosity levels `crit min any` to produce any one single graph that passed the criteria (all stages of the criteria, if applicable):



8. Six: Large Sample Space

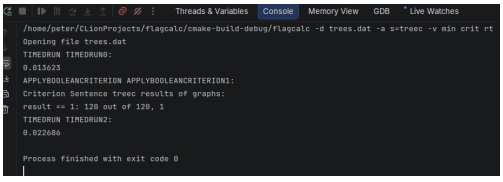
In large sample spaces, using minimal verbosity generally, one can come up with all manner of statistics of likelihoods. One can conjecture and, though not producing a proof, one can find if there are counter-examples by chance, and so on.

9. Seven: Graph Databases

10. Eight: Homemade Graph Databases

10.1. **A Database of Trees.** Easily brought about as in

```
./flagcalc -r 10 0.25 10000 -a s=treec -g o=trees.dat overwrite passed
```



Please consider how to bring about databases such as:

- Complete graphs (Hint: use `p=1.0`)
- Triangle-free graphs
- All graphs on five vertices (Hint: use `-f all` along with `-g o=graph5.fcg overwrite sorted`, the sorted being the key word; next, to accomplish the discover of "all" graphs, repeatedly run on as many random graphs at a time as your machine can handle, always feeding in the previous run's output file:
  - `./flagcalc -r 5 p=0.5 1000 -f all -g o=graph5.fcg overwrite sorted`
  - `./flagcalc -r 5 p=0.5 1000 -d graph5.fcg -f all -g o=graph5.fcg overwrite sorted`
  - Repeat. The actual number of graphs on five vertices is known, and equals 34. Repeat until 34 are obtained.
- All connected graphs on five vertices

- Check an online source for numbers of graphs per vertex count and, within reason, produce the database for each vertex count

11. Nine: Stored Procedures and Python Add-Ons

11.1. **Stored Procedures.** Any recurring motif in your queries may be candidate for turning into a "stored procedure" (a name borrowed from the SQL language). A "stored procedure" is simply a bit of Flagcalc Querying Language code given a name, given a possibly-empty set of parameters, and given its code. For example,

```
mtbool Bipartiteviacycles

FORALL (c IN Cycless, st(c) % 2 == 0)

END

mtbool Bipartiteviabipcrit

EXISTS (p IN Setpartition(V), st(p) == 2, bipc(p[0],p[1]))

END

mtbool Cycleoflength ( mtdiscrete n )

EXISTS (c IN Cycless, st(c) == n)

END
```

Here, the condition of being "bipartite" is the same as a graph having no odd cycles. *Bipartite* means the graph's vertices can be partitioned into two portions such that within either portion no two vertices have an edge between them. Classically, we have already seen the bipartite graph $K_{3,3}$ as `abc=def`.

To use this feature, run

```
./flagcalc -r 7 p=0.5 1000 -a isp=storedprocedures.dat
        s=Bipartiteviacycles -v i=minimal3.cfg
```

To check it is equivalent to internal criterion "`bipc`"

```
/home/peter/CLionProjects/flagcalc/cmake-build-debug/flagcalc -r 7 p=0.5 1000 -a isp=../scripts/storedprocedures.dat "s=Bipartiteviabipcrit IFF Bipartiteviacycles" s=Bipartiteviacycles -v i=minimal3.cfg
Opening file ../scripts/storedprocedures.dat
Opening file ../scripts/recursion.dat
Opening file ../scripts/planarity.dat
Opening file ../scripts/regular.dat
RANDOMGRAPHS : r1 random graph with edgecnt probability: 7, 10.500000, 1000
TIMEDRUN TIMEDRUN1000:
0.006102
APPLYBOOLEANCRITERION APPLYBOOLEANCRITERION1001:
Criterion Sentence Bipartiteviabipcrit IFF Bipartiteviacycles results of graphs:
result == 1: 1000 out of 1000, 1
APPLYBOOLEANCRITERION APPLYBOOLEANCRITERION1002:
Criterion Sentence Bipartiteviacycles results of graphs:
result == 0: 948 out of 1000, 0.948
result == 1: 52 out of 1000, 0.052
TIMEDRUN TIMEDRUN1003:
0.8947
TIMEDRUN TimedRunVerbosity:
0.002355

Process finished with exit code 0
```

Next, notice one additional stored procedures: `Cycleoflength`, which takes one input argument "`n`", and returns true or false as to whether a cycle of length n exists in the graph

(in the querying language, `st(c)` means "size tally of c; "tally" means it is a discrete or integer value, and size is the tally's name, here the very oft-used `st`).

Regarding Eulerian tours, we dually have a native set-valued measure "`nWalksvs`" that returns all n-walks starting and ending at the given two vertices; we also see how to code this as a stored procedure "`nwalkss`".

```
mttuple ncopies( mtdiscrete n, mtset S )

TUPLE (i IN NN(n), S)

END

mtset nwalkss( mtdiscrete n )

SETD (W IN Maps(n+1,V), FORALL (i IN n, ac(W[i],W[i+1])), W)

END

mtbool Eulertourc

EXISTS (W IN nWalksvs( edgecm, 0, 0 ), FORALL (e IN E, EXISTS (n IN edgecm, e == {W[n],W[n+1]})) AND W[0] == W[edgecm])

END

mtbool Eulertourc2

EXISTS (W IN nWalksvs( edgecm, 0, 0 ), st(SET (n IN edgecm, {W[n],W[n+1]})) == edgecm)

END
```

And this code in action

```
/home/peter/CLionProjects/flagcalc/cmake-build-debug/flagcalc -d f=abcd -a "e=BIGCUP (a IN V, b IN V, nWalksvs(3,a,b))" -v i=minimal3.cfg allsets
TIMEDRUN TIMEDRUN1:
0.000189
APPLYSETCRITERION APPLYSETCRITERION2:
Set type output, size == 108
{   Tuple type output, size == 4
    <3, 0, 1, 3>,
    Tuple type output, size == 4
    <3, 0, 2, 3>,
    Tuple type output, size == 4
    <3, 1, 0, 3>,
    Tuple type output, size == 4
    <3, 1, 2, 3>,
```

```
/home/peter/CLionProjects/flagcalc/cmake-build-debug/flagcalc -d f=abcd -a isp=../scripts/storedprocedures.dat "p=nwalksbetweenp( 3 )" -v i=minimal3.cfg allsets
Opening file ../scripts/storedprocedures.dat
Opening file ../scripts/recursion.dat
Opening file ../scripts/planarity.dat
Opening file ../scripts/regular.dat
TIMEDRUN TIMEDRUN1:
0.000191
APPLYTUPLECRITERION APPLYTUPLECRITERION2:
Tuple type output, size == 4
<   Tuple type output, size == 4
    <6, 7, 7, 7>,
    Tuple type output, size == 4
    <7, 6, 7, 7>,
    Tuple type output, size == 4
    <7, 7, 6, 7>,
    Tuple type output, size == 4
    <7, 7, 7, 6>
>
Count, average, min, max of tuple size Tuple-valued formula nwalksbetweenp( 3 ): 1, 4, 4, 4
TIMEDRUN TIMEDRUN3:
0.006037
TIMEDRUN TimedRunVerbosity:
6.8e-05

Process finished with exit code 0
```

See that the number of n-walks is also the sum of the matrix entries on the second screenshot (that is, a matrix fact is that the adjacency matrix raised to the n power gives an entry for every count of n-walks between the vertices expressed as row and column).

```
/home/peter/CLionProjects/flagcalc/cmake-build-debug/flagcalc -d "f=-abcdaefga -ahijc -ahikf" -a isp=../scripts/storedprocedures.dat "s=SETD (W IN nwalkss(5), W[0] == 0 AND W[5] == 0, W) == nWalksvs(5,0,0)" -v i=minimal3.cfg allsets
Opening file ../scripts/storedprocedures.dat
Opening file ../scripts/recursion.dat
Opening file ../scripts/planarity.dat
Opening file ../scripts/regular.dat
TIMEDRUN TIMEDRUN1:
0.000308
APPLYBOOLEANCRITERION APPLYBOOLEANCRITERION2:
Criterion Sentence SETD (W IN nwalkss(5), W[0] == 0 AND W[5] == 0, W) == nWalksvs(5,0,0) results of graphs:
result == 1: 1 out of 1, 1
TIMEDRUN TIMEDRUN3:
5.34169
TIMEDRUN TimedRunVerbosity:
8.3e-05

Process finished with exit code 0
```

11.2. **Python Add-Ons.** A Python add-on is just like a stored procedure, except you use the Python language to write your new measures.

For time-sensitive queries, or large datasets, any Python invocation inherits a design feature or flaw of Python itself, namely the GIL or global lock allowing only one CPU thread at a time. Flagcalc itself is highly multi-threaded (and constantly improving in this regard); if you are up against time constraints in your queries, coding add-ons in Python will drastically reduce your efficiency (thirty seconds on sixteen cores amounts to $30 * 16 = 480$ seconds, or eight minutes, on one thread)

That said, Python is a go-to language for many occasions. The included shell script `scripts/testsuitepython.sh` and the Python file `python/pymeas.py` provide some documentation or at least code to learn from.

12. Ten: Inventing New Randomizers and Native-Coded Measures

Here is a motivating snippet from Flagcalc's C++ code-base, `edgecm` ("edge count measure"):

```
749  class edgecnttally : public tally {
750  public:
751
752      int takemeas( neighborstype* ns, const params& ps ) {
753          return edgecnt(ns->g);
754      }
755      int takemeas( const int idx, const params& ps ) {
756          return edgecnt((*rec->gptrs)[idx]);
757      }
758      edgecnttally(mrecords* recin) : tally( recin, shortnamein: "edgecm", name: "Graph's edge count") {}
759  };
```

```
1514  int edgecnt( const graphtype* g ) {
1515      int res = 0;
1516      for (int n = 0; n < g->dim-1; ++n) {
1517          for (int i = n+1; i < g->dim; ++i) {
1518              if (g->adjacencymatrix[n*g->dim + i]) {
1519                  res++;
1520              }
1521          }
1522      }
1523      return res;
1524  }
```

Here is a portion of a (much) more sophisticated measure, `treec`:

```
class treecrit : public crit
{
public:
    treecrit(mrecords* recin ) : crit(recin, shortnamein: "treec", name: "Tree criterion") {}

    bool takemeas( neighborstype* ns, const params& ps ) override
    {
        auto g :graphtype*  = ns->g;
        int dim = g->dim;
        if (dim <= 0)
            return true;
        int* visited = (int*)malloc(size:dim*sizeof(int));

        for (int i = 0; i < dim; ++i)
        {
            visited[i] = -1;
        }

        visited[0] = 0;
        bool allvisited = false;
        while (!allvisited)
        {
            bool changed = false;
            for ( int i = 0; i < dim; ++i)
            {
                if (visited[i] >= 0)
                {
                    for (int j = 0; j < ns->degrees[i]; ++j)
                    {
                        vertextype nextv = ns->neighborslist[i*dim+j];
                        // loop = if a neighbor of vertex i is found in visited
                        // and that neighbor is not the origin of vertex i
                        bool loop = false;
                        if (visited[nextv] >= 0)
                            if (visited[nextv] != i)
                                if (visited[i] != nextv)
                                    loop = true;

                        if (loop)
                        {
                            free (visited);
```

Indeed, if you're comfortable with C++ coding, Flagcalc is open source and adheres to a distinction between *breadth* and *depth* enhancements: breadth means adding "natively coded" new C++ measures, or new randomizers (`-r`), or new verbosity levels (`-v`). Depth means actually adding features in the precise sense of overall reworking of the code-base.